

PREENTREGA JAVA: SISTEMA DE GESTIÓN DE ARTÍCULOS

Programación Orientada a Objetos — Herencia, polimorfismo, clases abstractas y método `toString()`

1. Objetivo del proyecto

Una empresa necesita desarrollar un sistema que permita administrar los artículos que comercializa. Actualmente trabaja con dos tipos de productos: **artículos electrónicos** y **artículos alimenticios**.

Aunque ambos tipos de artículos comparten información, como código, nombre, precio y categoría, cada uno posee características particulares.

El objetivo de esta preentrega es desarrollar desde cero una aplicación de consola en **Java**, aplicando los conceptos de Programación Orientada a Objetos (POO).

El sistema deberá permitir registrar, consultar, modificar, listar y eliminar artículos, utilizando `ArrayList` para almacenar la información.

Durante el desarrollo deberán aplicar los siguientes conceptos:

- Clases, objetos, atributos y métodos.
- Encapsulamiento, constructores, getters y setters.
- Herencia mediante `extends`.
- Clases y métodos abstractos.
- Polimorfismo.
- Sobrescritura de métodos mediante `@Override`.
- Utilización de `super` y `this`.
- Implementación del método `toString()`.
- Validaciones y estructuras de control.
- Manejo de listas mediante `ArrayList`.

2. Creación del proyecto

Crear un nuevo proyecto Java con el siguiente nombre:

`herencia_polimorfismo_tostring`

Dentro de la carpeta `src`, crear los siguientes paquetes:

- `com.techlab.articulo`
- `com.techlab.articulo.model`

La estructura final del proyecto deberá ser la siguiente:

```
None
herencia_polimorfismo_tostring
├── src
│   ├── com.techlab.articulo
│   │   ├── App.java
│   │   └── model
│   │       ├── Articulo.java
│   │       ├── ArticuloElectronico.java
│   │       ├── ArticuloAlimenticio.java
│   │       └── Categoria.java
```

Importante: Cada clase debe estar ubicada en el paquete indicado.

La clase `App` será responsable de ejecutar el programa, mientras que el paquete `model` contendrá las clases que representan los artículos y sus categorías.

3. Creación de las clases

3.1. Clase `Categoria`

Paquete: `com.techlab.articulo.model`

Crear una clase pública llamada `Categoria`.

Esta clase representa la categoría a la que pertenece un artículo.

Atributos:

- `codigo` (`int`): identificador de la categoría.
- `nombre` (`String`): nombre de la categoría.
- `descripcion` (`String`): descripción de la categoría.

Todos los atributos deberán declararse como `private`.

Implementar:

1. Un constructor que reciba los tres atributos.
2. Los getters y setters correspondientes.
3. El método `toString()`, sobrescrito mediante `@Override`, que permita mostrar la información completa de la categoría.

Ejemplo de visualización:

```
None
Categoria {
    codigo=1,
    nombre='Electrónica',
    descripcion='Productos tecnológicos y electrónicos'
}
```

3.2. Clase abstracta **Articulo**

Paquete: `com.techlab.articulo.model`

Crear una clase pública y abstracta llamada **Articulo**.

Esta será la clase padre de todos los artículos. Contendrá los atributos y comportamientos comunes que heredarán los distintos tipos de productos.

Atributos:

- `codigo` (`int`): identificador único del artículo.
- `nombre` (`String`): nombre del artículo.
- `precio` (`double`): precio del artículo.
- `categoria` (`Categoria`): categoría asociada al artículo.

Todos los atributos deberán ser `private`.

Implementar:

1. Un constructor que reciba `codigo`, `nombre`, `precio` y `categoria`.
2. Los getters y setters de todos los atributos.
3. Los siguientes métodos abstractos:

```
None
public abstract String getTipoArticulo();

public abstract String getDetalleEspecifico();
```

El método `getTipoArticulo()` deberá devolver el tipo de artículo.

El método `getDetalleEspecifico()` deberá devolver información particular de cada tipo de artículo.

Estos métodos serán implementados obligatoriamente por las clases hijas.

Método `toString()`

Sobrescribir el método `toString()` para devolver una representación legible del artículo, incluyendo:

- Código.
- Nombre.
- Precio.
- Nombre de la categoría.
- Tipo de artículo.
- Detalle específico.

Para obtener el tipo y el detalle, se deberán utilizar los métodos `getTipoArticulo()` y `getDetalleEspecifico()`.

Importante: Al ser abstracta, la clase `Articulo` no podrá instanciarse directamente mediante `new Articulo(...)`.

3.3. Clase `ArticuloElectronico`

Paquete: `com.techlab.articulo.model`

Crear una clase pública llamada `ArticuloElectronico`, que herede de `Articulo` mediante `extends`.

Atributo propio:

- `garantiaMeses (int)`: cantidad de meses de garantía del artículo.

Implementar:

1. Un constructor que reciba `codigo`, `nombre`, `precio`, `categoria` y `garantiaMeses`.
2. Utilizar `super(...)` para inicializar los atributos heredados.
3. Inicializar el atributo propio utilizando `this`.
4. Crear el getter y setter de `garantiaMeses`.
5. Sobrescribir `getTipoArticulo()` para devolver "Electrónico".
6. Sobrescribir `getDetalleEspecifico()` para devolver la garantía en meses.

7. Crear un método público llamado `nroTelMesaDeAyudaParaReclamos()` que devuelva el texto "0800-123-4567".
8. Sobrescribir `toString()` utilizando `super.toString()` y agregando al final el texto `[subtipo electrónico]`.

3.4. Clase `ArticuloAlimenticio`

Paquete: `com.techlab.articulo.model`

Crear una clase pública llamada `ArticuloAlimenticio`, que herede de `Articulo`.

Atributo propio:

- `diasParaVencimiento (int)`: cantidad de días restantes para el vencimiento del artículo.

Implementar:

1. Un constructor que reciba `codigo`, `nombre`, `precio`, `categoria` y `diasParaVencimiento`.
 2. Utilizar `super(...)` para inicializar los atributos heredados.
 3. Crear el getter y setter del atributo propio.
 4. Sobrescribir `getTipoArticulo()` para devolver "Alimenticio".
 5. Sobrescribir `getDetalleEspecifico()` para devolver los días restantes para el vencimiento.
 6. Sobrescribir `toString()` utilizando `super.toString()` y agregando al final `[subtipo alimenticio]`.
-

4. Clase principal `App`

Paquete: `com.techlab.articulo`

Crear una clase pública llamada `App` con el método principal:

```
None
public static void main(String[] args)
```

Desde esta clase se administrará toda la interacción con el usuario mediante la consola.

4.1. Inicialización

Dentro del método `main`:

1. Crear un objeto `Scanner` para leer datos ingresados por teclado.
2. Crear una lista `ArrayList<Articulo>` para almacenar los artículos.
3. Crear una lista `ArrayList<Categoria>` para almacenar las categorías.
4. Invocar un método que precargue las categorías.
5. Crear una variable `int opcion` para controlar el menú.
6. Implementar un ciclo `do-while` para mantener el programa en ejecución hasta que el usuario elija salir.
7. Utilizar un `switch` para ejecutar la opción seleccionada.
8. Cerrar el `Scanner` al finalizar.

4.2. Precarga de categorías

Crear el método:

```
None  
public static void precargarCategorias(  
    ArrayList<Categoria> categorias  
)
```

Este método deberá agregar las siguientes categorías a la lista:

Categoría 1

- Código: 1
- Nombre: `Electrónica`
- Descripción: `Productos tecnológicos y electrónicos`

Categoría 2

- Código: 2
- Nombre: `Periféricos`
- Descripción: `Accesorios para computadora`

Categoría 3

- Código: 3
- Nombre: `Alimentos`
- Descripción: `Productos alimenticios`

Categoría 4

- Código: 4
- Nombre: `Limpieza`
- Descripción: `Artículos de limpieza del hogar`

Las categorías estarán disponibles desde el inicio de la aplicación y no será necesario desarrollar su alta, modificación o eliminación.

5. Menú principal

Al ejecutar el programa, se deberá mostrar el siguiente menú:

```
None

=====
SISTEMA DE ARTÍCULOS - CLASE 4 (HERENCIA Y TO_STRING)
=====
1 - Ingresar artículo
2 - Listar artículos
3 - Consultar un artículo
4 - Modificar un artículo
5 - Eliminar un artículo
6 - Listar categorías
0 - Salir
=====
Ingrese una opción:
```

El menú deberá volver a mostrarse después de cada operación, excepto cuando el usuario seleccione la opción 0.

Si se ingresa una opción inexistente, el sistema deberá mostrar un mensaje indicando que no es válida.

6. Funcionalidades del sistema

6.1. Opción 1: Ingresar artículo

Crear el método:

```
None

public static void ingresarArticulo(
    Scanner scanner,
    ArrayList<Articulo> articulos,
    ArrayList<Categoria> categorias
```

)

El método permitirá registrar nuevos artículos.

Pasos que deberá realizar:

1. Mostrar un submenú para seleccionar el tipo de artículo:

None

```
--- INGRESAR ARTÍCULO ---  
1 - Artículo electrónico  
2 - Artículo alimenticio
```

Seleccione el tipo de artículo:

2. Validar que el usuario seleccione únicamente **1** o **2**. Si ingresa otro valor, deberá volver a solicitarlo.
3. Solicitar el código del artículo.
4. Verificar que no exista otro artículo con el mismo código. Si existe, mostrar un mensaje y cancelar el ingreso.
5. Solicitar el nombre del artículo, que no podrá estar vacío.
6. Solicitar el precio, que no podrá ser negativo.
7. Mostrar todas las categorías disponibles.
8. Solicitar el código de una categoría existente.
9. Solicitar el atributo específico según el tipo seleccionado:
 - **Electrónico:** garantía en meses.
 - **Alimenticio:** días para vencimiento.
10. Crear un objeto de la clase correspondiente.
11. Agregar el objeto a la lista **articulos**.
12. Mostrar un mensaje de confirmación y los datos del artículo creado.

Requisito de polimorfismo:

Aunque se creen objetos de clases diferentes, ambos deberán almacenarse en una variable declarada como **Articulo**.

Ejemplo:

None

```
Articulo articulo;
```

Según el tipo seleccionado, esa variable contendrá un `ArticuloElectronico` o un `ArticuloAlimenticio`.

Finalmente, el objeto se agregará a `ArrayList<Articulo>`.

6.2. Opción 2: Listar artículos

Crear el método:

```
None
public static void listarArticulos(
    ArrayList<Articulo> articulos
)
```

El método deberá:

1. Verificar si la lista está vacía.
2. Si está vacía, mostrar `No hay artículos cargados`.
3. En caso contrario, recorrer la lista utilizando un `for-each`.
4. Mostrar cada objeto mediante `System.out.println(articulo)`.

Importante: No se deberán mostrar los atributos manualmente, uno por uno. Se deberá aprovechar la implementación de `toString()`.

Como los artículos pertenecen a diferentes clases hijas, Java deberá ejecutar el método `toString()` correspondiente al objeto real.

6.3. Opción 3: Consultar un artículo

Crear el método:

```
None
public static void consultarArticulo(
    Scanner scanner,
    ArrayList<Articulo> articulos
)
```

El método deberá:

1. Verificar que existan artículos cargados.
2. Solicitar el código del artículo a consultar.
3. Buscar el artículo dentro de la lista.
4. Si no existe, mostrar `El artículo no existe`.
5. Si existe, mostrar todos sus datos utilizando `toString()`.

6. Mostrar adicionalmente el resultado de `getDetalleEspecifico()`.

Esta funcionalidad permitirá observar cómo el mismo método devuelve información diferente dependiendo de la clase del objeto.

6.4. Opción 4: Modificar un artículo

Crear el método:

```
None
public static void modificarArticulo(
    Scanner scanner,
    ArrayList<Articulo> articulos,
    ArrayList<Categoria> categorias
)
```

El método deberá permitir modificar un artículo existente.

Pasos:

1. Verificar que la lista no esté vacía.
2. Solicitar el código del artículo.
3. Buscar el artículo por su código.
4. Si no existe, informar al usuario.
5. Si existe, solicitar:
 - Nuevo nombre.
 - Nuevo precio.
 - Nueva categoría.
6. Actualizar los atributos comunes mediante sus setters.
7. Identificar el tipo real del artículo mediante `instanceof`.
8. Solicitar y modificar el atributo particular:
 - Si es electrónico, actualizar `garantiaMeses`.
 - Si es alimenticio, actualizar `diasParaVencimiento`.
9. Mostrar `Artículo modificado correctamente`.

Uso de `instanceof` y casting

Como los artículos se almacenan en una lista declarada con el tipo padre `Articulo`, será necesario identificar la clase concreta para acceder a sus métodos específicos.

Por ejemplo:

```
None
if (articulo instanceof ArticuloElectronico) {
```

```
ArticuloElectronico electronico =  
    (ArticuloElectronico) articulo;  
}
```

Luego se deberá utilizar el setter correspondiente.

No se permitirá modificar el código del artículo desde esta opción.

6.5. Opción 5: Eliminar un artículo

Crear el método:

```
None  
public static void eliminarArticulo(  
    Scanner scanner,  
    ArrayList<Articulo> articulos  
)
```

El método deberá:

1. Verificar que existan artículos.
2. Solicitar el código del artículo a eliminar.
3. Buscarlo dentro de la lista.
4. Si no existe, mostrar un mensaje informativo.
5. Si existe, eliminarlo mediante `remove()`.
6. Mostrar `Artículo eliminado correctamente`.

6.6. Opción 6: Listar categorías

Crear el método:

```
None  
public static void listarCategorias(  
    ArrayList<Categoria> categorias  
)
```

El método deberá recorrer la lista de categorías mediante un `for-each` y mostrar cada objeto utilizando su método `toString()`.

7. Métodos auxiliares obligatorios

Para evitar repetir código, deberán implementar los siguientes métodos dentro de la clase `App`.

7.1. Buscar un artículo por código

None

```
public static Artículo buscarArticuloPorCodigo(
    ArrayList<Articulo> articulos,
    int codigo
)
```

Deberá recorrer la lista y devolver el objeto cuyo código coincida con el recibido por parámetro.

Si no encuentra ningún artículo, deberá devolver `null`.

7.2. Buscar una categoría por código

None

```
public static Categoria buscarCategoriaPorCodigo(
    ArrayList<Categoria> categorias,
    int codigo
)
```

Deberá devolver la categoría correspondiente al código recibido. Si no existe, devolverá `null`.

7.3. Solicitar una categoría existente

None

```
public static Categoria pedirCategoriaExistente(
    Scanner scanner,
    ArrayList<Categoria> categorias
)
```

Deberá solicitar un código de categoría y comprobar que exista.

Si el código no corresponde a ninguna categoría, deberá volver a solicitarlo hasta obtener uno válido.

7.4. Leer un número entero

None

```
public static int leerEntero(  
    Scanner scanner,  
    String mensaje  
)
```

Deberá:

- Mostrar el mensaje recibido.
- Leer un valor por teclado utilizando `nextLine()`.
- Convertirlo a `int` mediante `Integer.parseInt()`.
- Controlar los errores de conversión mediante `try-catch`.
- Repetir la solicitud cuando el dato no sea válido.

7.5. Leer un entero no negativo

None

```
public static int leerEnteroNoNegativo(  
    Scanner scanner,  
    String mensaje  
)
```

Deberá utilizar `leerEntero()` y comprobar que el valor obtenido sea mayor o igual a cero.

Si el valor es negativo, deberá solicitarlo nuevamente.

7.6. Leer un número decimal no negativo

None

```
public static double leerDoubleNoNegativo(  
    Scanner scanner,  
    String mensaje  
)
```

Deberá:

- Leer el valor ingresado.
- Convertirlo mediante `Double.parseDouble()`.
- Controlar errores de conversión con `try-catch`.
- Rechazar valores negativos y valores no finitos, como `NaN` o `Infinity`.
- Repetir la solicitud hasta obtener un valor válido.

7.7. Leer texto no vacío

None

```
public static String leerTextoNoVacio(  
    Scanner scanner,  
    String mensaje  
)
```

Deberá solicitar un texto y verificar que no esté vacío ni contenga únicamente espacios.

Utilizar `trim()` para eliminar los espacios al principio y al final.

8. Ejemplo de funcionamiento

A continuación, se muestra una posible ejecución del programa.

None

```
SISTEMA DE ARTÍCULOS
```

```
1 - Ingresar artículo  
2 - Listar artículos  
3 - Consultar un artículo  
4 - Modificar un artículo  
5 - Eliminar un artículo  
6 - Listar categorías  
0 - Salir
```

```
Ingrese una opción: 1
```

```
--- INGRESAR ARTÍCULO ---
```

```
1 - Artículo electrónico  
2 - Artículo alimenticio
```

```
Seleccione el tipo de artículo: 1
```

```
Ingrese el código del artículo: 100  
Ingrese el nombre del artículo: Notebook  
Ingrese el precio del artículo: 850000
```

```
--- CATEGORÍAS DISPONIBLES ---
```

```
1 - Electrónica
```

```
2 - Periféricos
3 - Alimentos
4 - Limpieza
```

```
Ingrese el código de la categoría: 1
Ingrese la garantía en meses: 12
```

```
Artículo ingresado correctamente.
Resumen del objeto creado:
```

```
Articulo {
  codigo=100,
  nombre='Notebook',
  precio=850000.0,
  categoria='Electrónica',
  tipo='Electrónico',
  detalle='Garantía: 12 meses'
} [subtipo electrónico]
```

El formato presentado es un ejemplo del resultado esperado.

9. Validaciones obligatorias

El sistema deberá contemplar las siguientes situaciones:

Opción de menú inválida

Si el usuario ingresa una opción que no existe, el sistema deberá informar que la opción es incorrecta.

Tipo de artículo inválido

Si se ingresa un número distinto de 1 o 2, deberá solicitarse nuevamente.

Código de artículo duplicado

Si ya existe un artículo con el código ingresado, deberá cancelarse el alta y mostrar un mensaje.

Nombre vacío

Si el nombre está vacío o contiene solamente espacios, deberá solicitarse nuevamente.

Precio negativo

No se permitirá registrar un precio menor que cero.

Garantía negativa

No se permitirá ingresar una cantidad negativa de meses de garantía.

Días de vencimiento negativos

No se permitirá ingresar un número negativo de días.

Categoría inexistente

Si se ingresa un código de categoría que no existe, deberá solicitarse nuevamente.

Artículo inexistente

Si el código buscado no corresponde a ningún artículo, el sistema deberá informarlo.

Lista vacía

Si se intenta listar, consultar, modificar o eliminar cuando no hay artículos cargados, el sistema deberá mostrar un mensaje.

Formato incorrecto de números

Si se ingresa texto donde corresponde un número, deberá controlarse el error y solicitar nuevamente el dato.

10. Conceptos de POO que deben aplicarse

Herencia

`ArticuloElectronico` y `ArticuloAlimenticio` deberán heredar de `Articulo`.

Los atributos comunes no deberán repetirse en las clases hijas.

Abstracción

`Articulo` deberá declararse como una clase abstracta y definir los métodos abstractos `getTipoArticulo()` y `getDetalleEspecifico()`.

Polimorfismo

Se deberá utilizar una única lista:

None

```
ArrayList<Articulo> articulos =  
    new ArrayList<>();
```

La lista deberá permitir almacenar objetos de ambos tipos de artículos.

Sobrescritura de métodos

Las clases hijas deberán implementar los métodos abstractos utilizando `@Override` y sobrescribir `toString()`.

Encapsulamiento

Todos los atributos de las clases deberán ser privados y se accederá a ellos mediante getters y setters.

Uso de `super`

Las clases hijas deberán utilizar `super(...)` para llamar al constructor de `Articulo` y `super.toString()` para reutilizar su representación textual.

11. Orden recomendado de desarrollo

Para facilitar la construcción del proyecto desde cero, se recomienda respetar el siguiente orden:

1. Crear el proyecto y los paquetes.
 2. Crear la clase `Categoria` con sus atributos, constructor, getters, setters y `toString()`.
 3. Crear la clase abstracta `Articulo`.
 4. Crear las clases hijas `ArticuloElectronico` y `ArticuloAlimenticio`.
 5. Implementar los métodos abstractos y las sobrescrituras de `toString()`.
 6. Crear la clase `App` y su método `main`.
 7. Inicializar las listas y precargar las categorías.
 8. Implementar el menú principal.
 9. Desarrollar los métodos auxiliares de lectura y validación.
 10. Implementar los métodos de búsqueda.
 11. Implementar el ingreso y listado de artículos.
 12. Implementar la consulta, modificación y eliminación.
 13. Probar todas las opciones y validaciones.
-

12. Pruebas que deberán realizar

Antes de finalizar la preentrega, deberán comprobar que:

- Se puede ingresar un artículo electrónico.
 - Se puede ingresar un artículo alimenticio.
 - Ambos se almacenan en el mismo `ArrayList<Articulo>`.
 - El listado muestra correctamente los datos específicos de cada tipo.
 - No se permiten códigos duplicados.
 - La consulta encuentra artículos por código.
 - Se pueden modificar los atributos comunes y específicos.
 - Se puede eliminar un artículo.
 - Las categorías se muestran correctamente.
 - Las validaciones funcionan ante datos incorrectos.
 - El menú continúa funcionando hasta seleccionar `0`.
-

13. Condiciones de la preentrega

El proyecto deberá cumplir con las siguientes condiciones:

- Estar desarrollado en Java y ejecutarse por consola.
- Respetar los nombres de los paquetes, clases, atributos y métodos indicados en lo posible, se puede modificar.
- Utilizar `ArrayList` para almacenar los artículos y las categorías.
- Aplicar herencia y polimorfismo correctamente.
- Utilizar clases abstractas y `@Override`.
- Contar con todas las funcionalidades del menú de ser posible.
- Ejecutarse sin errores de compilación.

En esta etapa **no se deberán implementar interfaces, bases de datos, archivos de persistencia ni una interfaz gráfica**. Toda la información se almacenará en memoria mientras el programa esté en ejecución.

14. Resultado esperado

Al finalizar la preentrega, deberán haber construido un sistema funcional de gestión de artículos que permita realizar las operaciones de alta, consulta, modificación, listado y eliminación, aplicando los principios de Programación Orientada a Objetos.

El objetivo no es solamente lograr que el programa funcione, sino comprender cómo la herencia permite reutilizar atributos y métodos, cómo el polimorfismo permite

trabajar con diferentes tipos de objetos y cómo `toString()` facilita su representación.

El desarrollo deberá realizarse desde cero, respetando la estructura y los requisitos especificados en este enunciado.